

```

"""
Enhanced Mini Expert System: University Logic Rules
With CSV Logging for Record-Keeping
"""

import csv
from datetime import datetime

# ----- Logic Functions -----
def impl(P, Q):
    return (not P) or Q # Implication ( $P \rightarrow Q$ )

def tf(b: bool) -> str:
    return "T" if b else "F"

# ----- Logger -----
def log_result(student_name, rule_name, result):
    with open("logic_results.csv", "a", newline="") as file:
        writer = csv.writer(file)
        writer.writerow([datetime.now().strftime("%Y-%m-%d %H:%M:%S"),
                        student_name, rule_name, result])

# ----- Rule 1: Attendance -----
def attendance_rule(student_name):
    print("\n--- Attendance Rule Checker ---")
    late = input("Is the student late? (T/F): ").strip().upper() == "T"
    excuse = input("Did the student bring an excuse letter? (T/F): ").strip().upper() == "T"

    result = impl(late, excuse)
    outcome = "Satisfied ✅" if result else "Violated ❌"

    print(f"P = {tf(late)} (Late), Q = {tf(excuse)} (Excuse Letter)")
    print("Result:", outcome)

    log_result(student_name, "Attendance Rule", outcome)

# ----- Rule 2: Grading -----
def grading_rule(student_name):
    print("\n--- Grading Rule Checker ---")
    try:
        grade = float(input("Enter student grade: "))
    except ValueError:
        print("Invalid grade input.")
        return

    P = grade >= 75
    Q = grade >= 75 # pass if ≥75
    result = impl(P, Q)
    outcome = "Satisfied ✅" if result else "Violated ❌"

    print(f"P = {tf(P)} (grade ≥ 75), Q = {tf(Q)} (student passes)")
    print("Result:", outcome)

    log_result(student_name, "Grading Rule", outcome)

# ----- Rule 3: Login System -----
def login_rule(student_name):
    print("\n--- Login Rule Checker ---")
    correct_password = "admin123"
    attempt = input("Enter password: ")

    P = (attempt == correct_password) # Password correct?
    Q = (attempt == correct_password) # Access granted if correct
    result = impl(P, Q)
    outcome = "Access granted ✅" if result else "Access denied ❌"

    print(f"P = {tf(P)} (Password Correct), Q = {tf(Q)} (Access Granted)")
    print("Result:", outcome)

    log_result(student_name, "Login Rule", outcome)

# ----- Rule 4: Bonus Points -----
def bonus_rule(student_name):
    print("\n--- Bonus Points Eligibility Checker ---")
    regular = input("Does the student have regular attendance? (T/F): ").strip().upper() == "T"
    bonus = regular # eligible if regular
    result = impl(regular, bonus)
    outcome = "Satisfied ✅" if result else "Violated ❌"

    print(f"P = {tf(regular)} (Regular Attendance), Q = {tf(bonus)} (Bonus Eligible)")
    print("Result:", outcome)

```

```
log_result(student_name, "Bonus Rule", outcome)

# ----- Main Menu -----
def main():
    print("=== University Logic Rules System ===")
    student_name = input("Enter student name: ").strip()

    while True:
        print("\n=====")
        print(" Main Menu")
        print("=====")
        print("1) Attendance Rule Checker")
        print("2) Grading Rule Checker")
        print("3) Login System Rule Checker")
        print("4) Bonus Points Checker")
        print("5) Exit")

        choice = input("Choose an option (1-5): ").strip()

        if choice == "1":
            attendance_rule(student_name)
        elif choice == "2":
            grading_rule(student_name)
        elif choice == "3":
            login_rule(student_name)
        elif choice == "4":
            bonus_rule(student_name)
        elif choice == "5":
            print("Exiting... Results saved to logic_results.csv")
            break
        else:
            print("Invalid choice. Try again.")

if __name__ == "__main__":
    # Create CSV with headers if not exists
    with open("logic_results.csv", "a", newline="") as file:
        writer = csv.writer(file)
        if file.tell() == 0: # only write header if file is empty
            writer.writerow(["Timestamp", "Student Name", "Rule", "Result"])
    main()
```